



Bài 6 Cấu trúc dữ liệu và giải thuật 2

Công nghệ thông tin (Đại học Xây dựng Hà Nội)



Scan to open on Studocu

Bài 6

Chương 4. ĐỒ THỊ - GRAPH

4.1. Khái niệm

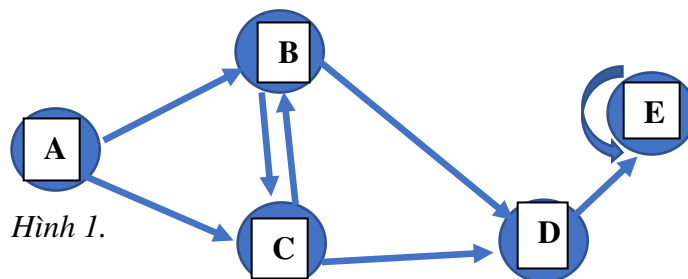
Cấu trúc dữ liệu đồ thị (graph) là kiểu cấu trúc tổng quát hơn cấu trúc cây TREE đã học. Sơ đồ mạng lưới (network) là một dạng ứng dụng biến thể của đồ thị.

Một đồ thị ở dạng biểu diễn hình ảnh là các cặp nút và cạnh nối với nhau.

Một đồ thị ở dạng biểu diễn toán học là một tập hợp (V, E) , trong đó V (vertices) là tập các nút và E (edges) là tập các cạnh nối các cặp nút.

Một đồ thị mà các cạnh nối nút không có hướng gọi là đồ thị vô hướng, các cạnh có hướng gọi là đồ thị có hướng.

Ví dụ, trong đồ thị ở hình 1 dưới đây, tập nút $V = \{A, B, C, D, E\}$, tập các cạnh $E = \{AB, AC, BC, BD, CD, DE\}$



Hình 1.

Các khái niệm :

- Nút kề nhau : 2 nút được nối với nhau bằng 1 cạnh. Ví dụ A kề B, A kề C,..., D kề E.
- Đường : biểu diễn một dãy các cạnh nối liên các nút. Trong hình trên, ACDE hoặc ABDE biểu diễn 2 đường đi từ A tới E.

4.2. Đồ thị có hướng

4.2.1. Khái niệm

Khi các nút nối với nhau bằng các cạnh có hướng thì ta có một đồ thị có hướng. Kiểu cấu trúc cây – Tree – đã xét trước đây là một dạng riêng của đồ thị có hướng, trong cấu trúc cây, các nút luôn xét theo hướng đi từ nút gốc đến các nút lá. Trong đồ thị trên, A là nút gốc, E là nút lá và có 3 đường có thể đi từ A $\rightarrow$ E là ABDE, ACDE, ABCDE. Khác với Cây, trong đồ thị có hướng, có thể ở một nút có hướng quay vòng tại nút đó (giống hình ảnh một bùng binh, vòng xoay ở nút giao thông có nhiều đường giao nhau).

Đồ thị có hướng được áp dụng nhiều trong thực tế, tạo thành một lĩnh vực riêng là sơ đồ mạng lưới, được vận dụng giải các bài toán như quản lý thi công các công trình xây dựng, quản lý mạng lưới điện, lập kế hoạch sản xuất kinh doanh của các doanh nghiệp...

4.2.2. Cài đặt đồ thị có hướng

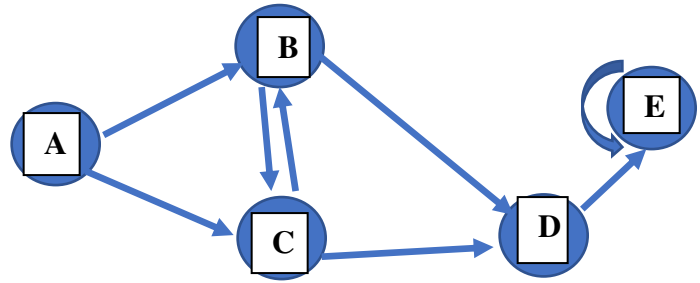
a) Ma trận kề - Adjacency matrix

Cách cài đặt thông dụng là dùng ma trận kề của một đồ thị có n nút, là một ma trận vuông A có n hàng n cột $A[n][n]$, trong đó :

$$A[i][j] = \begin{cases} 1 & \text{nếu có cạnh nối nút } i \text{ đến nút } j \\ 0 & \text{nếu không có cạnh nối nút } i \text{ đến nút } j \end{cases}$$

A – Adjacency matrix – ma trận kề

Như vậy, với đồ thị hình 1 ta lập ma trận kề :

$$A[] = \begin{array}{c|ccccc} & A & B & C & D & E \\ \hline A & 0 & 1 & 1 & 0 & 0 \\ B & 0 & 0 & 1 & 1 & 0 \\ C & 0 & 1 & 0 & 1 & 0 \\ D & 0 & 0 & 0 & 0 & 1 \\ E & 0 & 0 & 0 & 0 & 1 \end{array}$$


b) Mảng giá trị các nút

Dùng mảng D[] lưu dữ liệu các nút, D[i] là dữ liệu của nút i.

c) Danh sách kề

Trong ma trận kề thường có nhiều phần tử = 0, nếu lưu trữ tất cả sẽ lãng phí ô nhớ. Có cách khác lưu trữ thông tin của ma trận kề tiết kiệm bộ nhớ bằng cách dùng danh sách liên kết theo hàng, mỗi danh sách nút theo hàng sẽ là một mảng liên kết, được duy trì bởi một con trỏ trỏ đến nút đầu của mảng nút đó, mỗi phần tử tiếp theo là một nút có 2 thành phần : dữ liệu và liên kết, được nối tiếp ra từ nút đầu của mảng nút.

Ký hiệu V[x] là con trỏ cho nút x.

V[A] -> [A | next] -> [B | next] -> [C | next] -> [D | next] -> [E | NULL]

V[B] -> [B | next] -> [C | next] -> [D | NULL] -> [E | NULL]

V[C] -> [C | next] -> [D | NULL] -> [E | NULL]

V[D] -> [D | next] -> [E | NULL]

V[E] -> [E | NULL]

Nếu thay chữ cái tên nút bằng số thứ tự nút 1,2,... thì ta danh sách kề có dạng :

V[1] -> [1 | next] -> [2 | next] -> [3 | next] -> [4 | next] -> [5 | NULL]

V[2] -> [2 | next] -> [3 | next] -> [4 | NULL] -> [5 | NULL]

V[3] -> [3 | next] -> [4 | NULL] -> [5 | NULL]

V[4] -> [4 | next] -> [5 | NULL]

V[5] -> [5 | NULL]

Hình 2. Danh sách kề

Để thiết lập danh sách liên kế như hình 2 trên, có các bước :

1) Khai báo mảng các con trỏ $V[]$, mỗi phần tử có 2 thành phần du_lieu và $next$

- n là số nút tối đa của đồ thị

2) Khởi tạo nút đầu tiên của nút i có dữ liệu, $next$ được gán NULL

{ $V[i].du_lieu = du_lieu0$; $V[i].next = NULL$ }

Với mỗi nút i sẽ thêm các nút kế tiếp theo sơ đồ như hình 2 cho đến hết, sử dụng hàm thêm nút vào cuối danh sách liên kết đã học ở bài 2.

3) Tăng i và lặp lại bước 2 cho đến khi $i=n$.

4.2.3. Duyệt tìm theo chiều sâu trên đồ thị có hướng

Ý tưởng của duyệt tìm theo chiều sâu

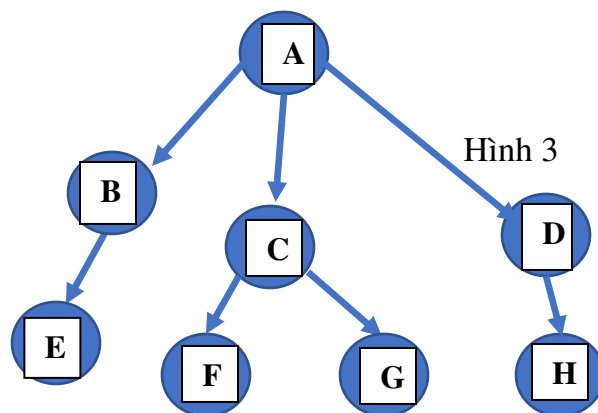
là từ gốc đi xuống nút con, thăm nút con của nút con...cho đến khi hết các cấp dưới của nút con mới quay lên duyệt nút con khác cho đến khi tìm được.

Với đồ thị ở hình 3, đầu tiên thăm nút A,

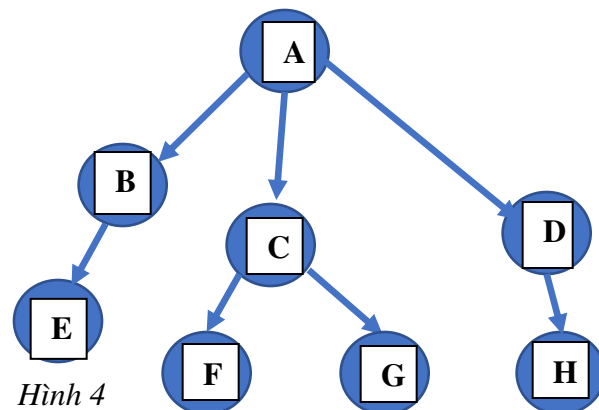
sau đó thăm nút con B, thăm nút con E,

quay lên thăm nút con C, thăm tiếp các nút con của C là F,G; quay lên thăm nút D, thăm tiếp nút con của D là H.

Thông thường dùng cấu trúc ngăn xếp – stack để tổ chức duyệt đồ thị theo chiều sâu.



Hình 3



Hình 4

5.2.4. Duyệt tìm theo chiều ngang trên đồ thị có hướng

Theo cách duyệt này ta đi theo chiều ngang của từng cấp.

Ví dụ : với đồ thị ở hình 4, đầu tiên đi từ gốc A,

sau đó thăm các nút mức 1 có 3 nút B, C, D

sau đó thăm các nút mức 2 có 4 nút E, F, G, H.

Cách này giống như duyệt hết các nút ở mức thứ i rồi mới duyệt ở mức thứ $i+1$ trong cây nhị phân.

Thông thường dùng cấu trúc hàng đợi – queue để tổ chức duyệt đồ thị theo chiều ngang

4.3. Đồ thị vô hướng

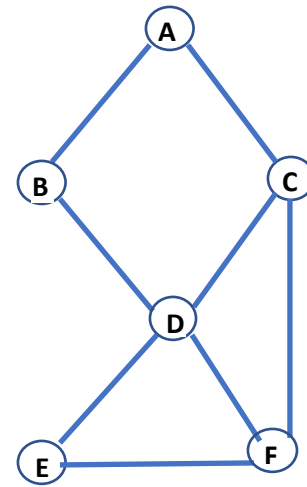
Khi các cạnh trong đồ thị không chỉ hướng thì ta có đồ thị vô hướng. Trong đồ thị vô hướng không có một nút nào được nối với chính nó (*không có xoay vòng*).

Cài đặt đồ thị vô hướng cũng gần giống như đối với đồ thị có hướng, dùng ma trận kề, danh sách các cạnh.

4.3.1. Cài đặt đồ thị vô hướng bằng ma trận kề

Cho đồ thị như hình 5.

Ma trận kề bây giờ có dạng :

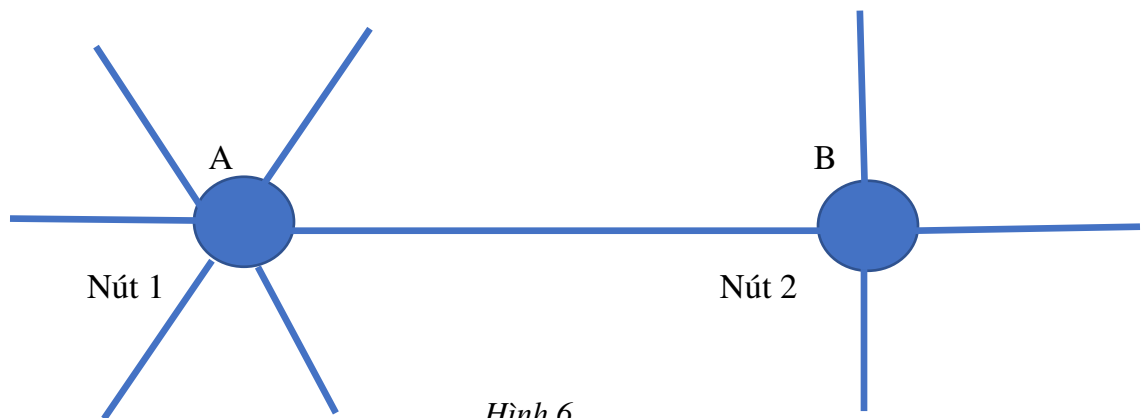
$$A[] = \begin{array}{c|cccccc} & A & B & C & D & E & F \\ \hline A & 0 & 1 & 1 & 0 & 0 & 0 \\ B & 1 & 0 & 0 & 1 & 0 & 0 \\ C & 1 & 0 & 0 & 1 & 0 & 1 \\ D & 0 & 1 & 1 & 0 & 1 & 1 \\ E & 0 & 0 & 0 & 1 & 0 & 1 \\ F & 0 & 0 & 1 & 1 & 1 & 0 \end{array}$$


Hình 5

Nhận xét : *ma trận kề của đồ thị vô hướng luôn đối xứng qua đường chéo chính.*

4.3.2. Cài đặt đồ thị vô hướng bằng danh sách các cạnh

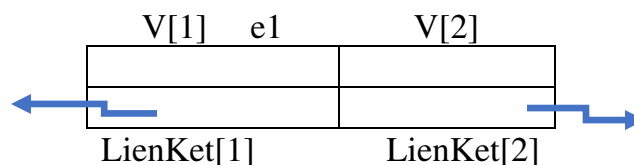
Trong đồ thị vô hướng, mỗi nút ở đầu cạnh có thể nối với nhiều nút khác. Chẳng hạn khi biểu diễn mạng lưới các đường phố của một khu phố, có những đoạn đường mà một đầu là ngã 6, một đầu là ngã tư :



Hình 6.

Danh sách các cạnh là khi ta đánh số thứ tự các nút, mỗi cạnh sẽ được lưu trữ thông tin dạng CẠNH – NÚT (Edge Node), sẽ lưu nút $V[i]$ mà cạnh được nối tới. Trong hình 6 trên, cạnh nối A-B là cạnh nối nút $V[1]$ và $V[2]$; đồng thời phải hiểu $V[1]$ là điểm cuối của cạnh khác và $V[2]$ cũng vậy. Ta dùng mảng các con trỏ $V[i]$, mỗi con trỏ $V[i]$ trỏ đến nút đầu lưu trữ dữ liệu của các nút, nút đầu $V[i]$ này có trường liên kết trỏ đến một cạnh- nút mà có nút i là một trong các điểm cuối của nó.

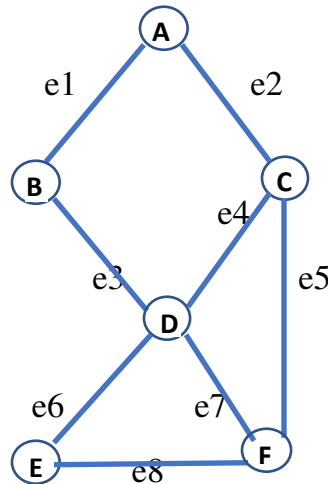
Mỗi phần tử e_i trong danh sách các cạnh có dạng :



Khi cạnh với tính cách là 1 phần tử trong danh sách liên kết thì nó giống như 1 nút trong danh sách liên kết đôi, có 2 con trỏ trái và phải liên kết đến nút bên trái và nút bên phải.

Với đồ thị ở hình 5 trên, ta có danh sách cạnh như sau:

- 1.AB
- 2.AC
- 3.BD
- 4.CD
- 5.CE
- 6.DE
- 7.DF
- 8.EF



Hình 7

Chú ý : một cạnh đã được kê khai thì không kê khai lại, ví dụ đã khai báo cạnh AB thì không khai BA nữa.

4.4.Các thuật toán cơ bản trên đồ thị

Trong lý thuyết đồ thị, có 2 kiểu duyệt cơ bản là duyệt theo chiều ngang và duyệt theo chiều sâu. Duyệt đồ thị theo chiều ngang (*breadth-first searching* BFS) gần giống với duyệt cây nhị phân theo các mức trong cây, duyệt đồ thị theo chiều sâu (*depth-first searching* DFS) gần giống với duyệt cây theo tiền tự DLR; Chỉ có chỗ khác biệt là trong đồ thị không có nút gốc cố định, việc duyệt có thể bắt đầu từ một nút bất kỳ.

4.4.1.Thuật toán duyệt đồ thị theo bề ngang – BFS

Ý tưởng của thuật toán duyệt đồ thị theo bề ngang là duyệt các nút của đồ thị theo các mức theo khoảng cách xa dần nút gốc(nút được khởi đầu duyệt), mức sau cách mức trước 1 cạnh tính từ nút bắt đầu.

Giải thích thuật ngữ : trong các thuật toán BFS và DFS có sử dụng cụm từ “đã duyệt”, một số tài liệu dùng với ý nghĩa không rõ ràng, dễ nhầm lẫn. Trong bài này nói “đã duyệt” là đã có một tác động vào nút đó, chẳng hạn đã đánh dấu “đã duyệt”, đang thực hiện lập tập nút liền kề của nó, xét xem các nút trong tập liền kề của nó đã được duyệt chưa để đưa vào hàng đợi...

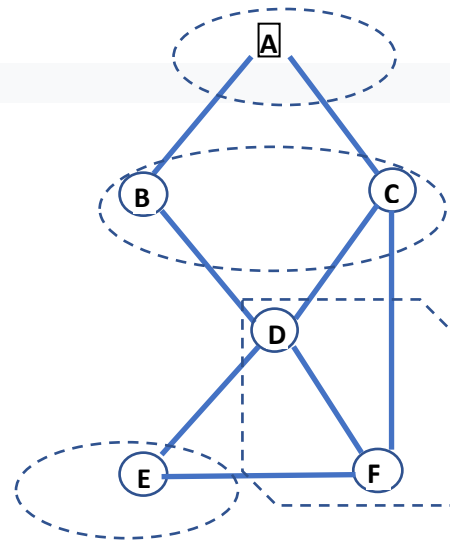
4.4.1.1.Giới thiệu thuật toán

Xét bài toán : cho một đồ thị vô hướng không có trọng số $G=(V,E)$, trong đó V là tập các nút, E là tập các cạnh; có một nút khởi đầu $s \in V$.

Để duyệt đồ thị theo BFS, sẽ sử dụng cấu trúc hàng đợi $Q[]$ và danh sách chưa được sắp là dãy $bfs[]$ cài đặt kiểu mảng array. Hàng đợi $Q[]$ dùng cho việc thăm qua các nút, lưu các nút đã được thăm và chờ đợi duyệt, nút nào được đánh dấu “đã duyệt” sẽ xóa khỏi hàng đợi rồi sau đó mới thực hiện các thao tác liên quan đến nó. Mảng $bfs[]$ để lưu kết quả, chỉ số i của mảng $bfs[i]$ là số thứ tự duyệt của nút, còn giá trị của $bfs[i]$ sẽ là số hiệu nút, ví dụ $bfs[6] = E$ có nghĩa là nút E có thứ tự duyệt là 6. Khởi đầu, chưa duyệt nút nào, gán tất cả các $bfs[i] = 0$, sau đó lần lượt các lần duyệt $i=1, bfs[1] = <\text{nút duyệt thứ nhất}>$, $i=2, bfs[2]= <\text{nút duyệt thứ 2}>$,....

Cho một đồ thị như sau :

Hình 8



Thuật toán duyệt đồ thị theo bề ngang là duyệt các nút của đồ thị theo các mức theo khoảng cách xa dần nút gốc (nút được khởi đầu duyệt), mức thứ $i+1$ cách xa nút gốc hơn mức thứ i một cạnh. Với đồ thị ở hình 8, mức 1 chỉ có nút khởi đầu A; mức 2 có nút B và C, mức 3 có nút D và F, mức 4 có một nút E.

4.4.1.2. Các bước của thuật toán BFS

Đầu vào : đồ thị $G=(V,E)$, V là tập các nút, E là tập các cạnh.

Đầu ra : đồ thị $G_{\text{kết quả}}=(V,E)$, với điều kiện mỗi nút của đồ thị kết quả được nối với nút khởi đầu **s** khi và chỉ khi nó đã được đánh dấu “đã duyệt”.

1. Thiết lập hàng đợi $Q[]$ và mảng $\text{bfs}[]$, khởi tạo nút s đưa vào Q ;

Mảng $\text{bfs}[]$ được dùng để lưu thứ tự duyệt các nút, $\text{bfs}[i] = \langle \text{số hiệu nút} \rangle$, nghĩa là chỉ số i của dãy $\text{bfs}[i]$ là thứ tự duyệt các nút, được ngầm gán từ $1 \div n$, n là số nút, còn giá trị của $\text{bfs}[i]$ là tên nút A/B/C/D/E/F, lúc đầu sẽ khởi tạo $\text{bfs}[1]=\dots=\text{bfs}[n]=0$. Ví dụ, $\text{bfs}[1]=A$ được hiểu là nút A duyệt đầu tiên, hay $\text{bfs}[1]=C$ nghĩa là nút duyệt đầu tiên là nút C. Trong đồ thị vô hướng, thuật toán duyệt theo bề ngang BFS có thể bắt đầu duyệt từ một nút bất kỳ, nút đó được lưu ở $\text{bfs}[1]$.

Thuật toán BFS được khái quát như sau:

1. Nút s được chọn khởi đầu duyệt, ký hiệu là u là nút đầu của mỗi bước lặp duyệt, nên bắt đầu duyệt sẽ coi $u=s$ và u được đưa vào hàng đợi $Q[]$. Trong $\text{bfs}[]$ chưa có nút nào được duyệt; ký hiệu i là thứ tự duyệt của 1 nút, k là thứ tự bước lặp thuật toán, lúc đầu gán $i=0$; $k=0$;

2. **while** (Q không rỗng)

{ Lặp lần thứ $k=k+1$;

Kiểm tra nút u đã duyệt chưa, nếu $\text{bfs}[i] = u$ là đã duyệt thì trở lại đầu vòng lặp **while** :

Nếu u chưa duyệt thì duyệt u , đánh dấu u đang duyệt, thứ tự duyệt $i=i+1 \Rightarrow \text{bfs}[i]=u$;

Tạo danh sách các nút liền kề $V\{u\}$, kiểm tra đối với mọi $v \in V\{u\}$ chưa duyệt thì đưa v vào $Q[]$;

Nếu đã xét hết các nút liền kề trong $V\{u\}$, thì xóa u khỏi đầu hàng đợi $Q[]$.

Kiểm tra nếu Q chưa rỗng thì lặp lại **while**.

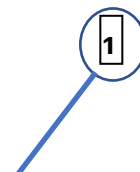
}

Với đồ thị ở hình 8, từng bước thực hiện như sau.

Tạo dãy $\text{bfs}[6]$ và hàng đợi $Q[6]$

Bước 0. Đưa nút khởi đầu A vào Q

i	0				
bfs[i]					



Q

A					
---	--	--	--	--	--

Tiếp theo thực hiện các bước lặp trong vòng lặp **while** cho đến khi hàng đợi Q[] rỗng :

while (!Empty(Q[])){Trong vòng lặp này thực hiện các lệnh :

- Lấy ra nút đầu Q đặt là u (chưa xóa u ở hàng đợi) ; nút khởi đầu là u=A.
- Xóa u khỏi hàng đợi Q[];
- Kiểm tra xem u đã duyệt chưa, nếu đã duyệt (đã có bfs[i]=u) thì bỏ qua các phần sau, lặp lại **while**
- Duyệt u, đánh dấu u đang duyệt, thứ tự duyệt i=i+1 => bfs[i]=u;
- Tạo tập các nút liền kề với u dưới dạng danh sách lienke(u);
- Lần lượt xét từng nút v thuộc lienke(u), nếu v chưa được duyệt (chưa có bfs[j]=v; j=1÷i) sẽ đưa v vào Q[];

}

Lần lặp 1. k=k+1=1; Lấy nút A đầu hàng đợi, duyệt nút A,

Xóa nút A khỏi hàng đợi Q[];

Kiểm tra xem u đã duyệt chưa : nếu đã duyệt thì bỏ qua các phần sau, lặp lại **while**

ở đây các bfs[]=0 , chưa duyệt.

i=i+1, ghi nút A vào thứ tự duyệt i=1 vào bfs[1]=A,

i	1				
bfs[i]	A	0	0	0	0

Tạo tập các nút liền kề với nút A: lienke(A)={B,C} (nút A nối với nút B , C).

Lần lượt xét 2 nút B,C này, chúng chưa được duyệt vì chưa có mặt trong bfs[] nên đưa 2 nút này vào hàng đợi Q[], vẽ thêm nút B, C vào cây :

Q

B	C				
---	---	--	--	--	--

Kiểm tra, Q chưa rỗng nên lặp tiếp vòng **while**.

Lần lặp 2. k=k+1=2. Lấy nút B ở đầu hàng đợi Q[]. Duyệt nút B, xóa nút B khỏi hàng đợi Q[];

Q

C					
---	--	--	--	--	--

Kiểm tra xem nút B đã duyệt chưa, nếu đã duyệt thì bỏ qua các phần sau, lặp lại **while**.

Nhưng nút B chưa duyệt => Ghi nút B vào thứ tự duyệt i=i+1=2 => bfs[i] => bfs[2]= B;

i	1	2			
bfs[i]	A	B			

Tạo tập các nút liền kề lienke(B)={D}, nút B nối với nút D, thêm nút D vào cây :

Xét nút D trong danh sách kề lienke(B), nó chưa được duyệt nên

đưa D vào hàng đợi Q[]:

Q

C	D				
---	---	--	--	--	--

Kiểm tra, Q chưa rỗng nên lặp tiếp vòng **while**.

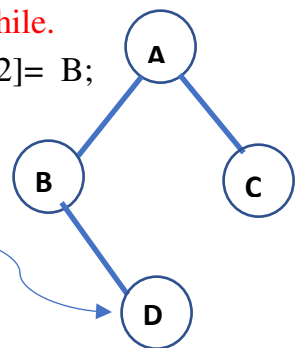
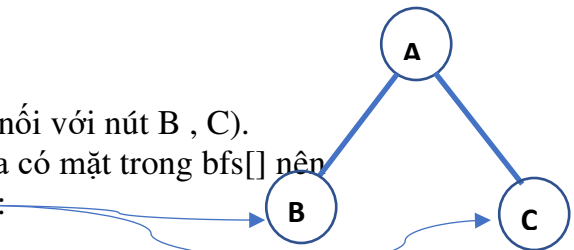
Lần lặp 3. k=k+1=3; Lấy nút C từ Q[]. Duyệt nút C, xóa nút C khỏi Q[].

Q

D					
---	--	--	--	--	--

Kiểm tra xem u=C đã duyệt chưa, nếu đã duyệt thì bỏ qua các phần sau, ,

lặp lại **while**, nhưng nút C chưa duyệt => i =i+1=3, ghi bfs[i]= bfs[3]= nút C;



i	1	2	3		
bfs[i]	A	B	C	0	0

Tạo tập các nút liền kề $lienke(C)=\{D,F\}$, nút C nối với nút D,F; nút D đã có trong cây nên chỉ thêm nút F vào cây:

Xét nút D,F trong danh sách kề $lienke(C)$, chúng chưa được duyệt nên đưa D,F vào Q[]:

Q [D | F | |]

Trên đồ thị dùng nét đứt nối C-D còn nút F bây giờ mới xét đến nên cạnh C-F vẽ nét liền.

Đã xét tất cả các nút $v \in lienke(C)$.

Kiểm tra, Q chưa rỗng nên lặp tiếp vòng **while**

Lần lặp 4. $k=k+1=4$; Lấy nút D từ Q[]. Duyệt nút D, xóa nút D khỏi Q[].

Q [| F | |]

Kiểm tra xem $u=D$ đã duyệt chưa, nếu đã duyệt thì bỏ qua phần sau, lặp lại **while**,

Nhưng nút D chưa duyệt $\Rightarrow i=i+1=4$, ghi nút D vào thứ tự duyệt $i=4 \Rightarrow bfs[4]=D$;

i	1	2	3	4	
bfs[i]	A	B	C	D	

Tạo tập các nút liền kề $lienke(D)=\{B,C,E,F\}$,

chỉ xét nút E,F trong danh sách này vì chúng chưa được duyệt và đưa chúng vào Q[].

Cạnh D-E vẽ nét liền, còn D-F dùng nét đứt nối vì cạnh C-F đã nối liền từ nút C.

Q [F | E | F |]

Đã xét tất cả các nút $v \in lienke(D)$.

Kiểm tra, Q chưa rỗng nên lặp tiếp vòng **while**

Lần lặp 5. $k=k+1=5$; Lấy nút F từ Q[]. Duyệt nút F, xóa nút F khỏi Q[] :

Q [E | F |]

Kiểm tra xem nút F đã duyệt chưa, nếu đã duyệt thì bỏ qua các phần sau, lặp lại **while**.

Nhưng nút F chưa duyệt $\Rightarrow i=i+1=5$; Ghi nút F vào thứ tự duyệt $i=5 \Rightarrow bfs[5]=F$;

i	1	2	3	4	5
bfs[i]	A	B	C	D	F

Tạo tập các nút liền kề $lienke(F)=\{E,D,C\}$.

Xét các nút trong danh sách kề $lienke(F)$, chỉ có nút E chưa duyệt nên đưa E vào Q[].

Q [E | F | E |]

Nút F có cạnh nối F-E, F-D, F-C, nút C, D đã duyệt, nút E đã được nối từ nút D, nên cạnh F-E ta dùng nét đứt.

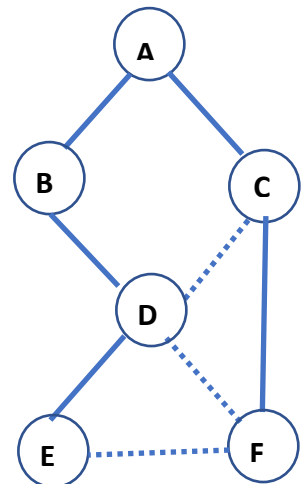
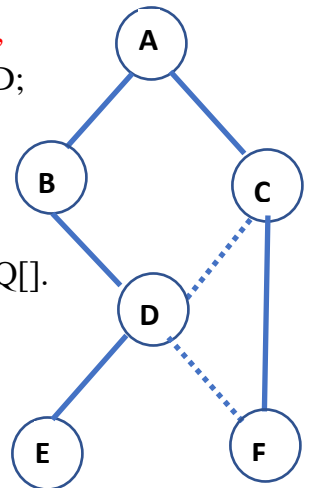
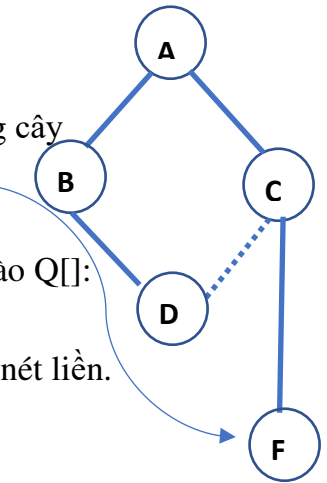
Đã xét tất cả các nút $v \in lienke(F)$.

Kiểm tra, Q chưa rỗng nên lặp tiếp vòng **while**

Lần lặp 6 : $k=k+1=6$; Lấy nút E từ Q[]. Duyệt nút E, xóa nút E khỏi Q[].

Q [F | E |]

Kiểm tra xem E đã duyệt chưa, nếu đã duyệt thì bỏ qua các phần sau, lặp lại **while**,



Vì nút E chưa duyệt => $i=i+1=6$; Ghi nút E vào thứ tự duyệt $i=6 \rightarrow \text{bfs}[6]=E$;

i	1	2	3	4	5	6
bfs[i]	A	B	C	D	F	E

Tạo tập các nút liền kề $\text{lienke}(E)=\{D,F\}$.

Xét tất cả các nút $v \in \text{lienke}(E)$ đã duyệt hết nên không thêm vào Q[]

Kiểm tra, Q chưa Rỗng nên lặp tiếp vòng **while**.

Lần lặp 7 : $k=k+1=7$; Lấy nút F từ Q[], xóa nút F khỏi Q[].

Q [E]

Kiểm tra nút F đã duyệt nên bỏ qua các phần sau, lặp lại **while**.

Kiểm tra, Q chưa Rỗng nên lặp vòng **while**

Lần lặp 8 : $k=k+1=8$; Lấy nút E từ Q[], xóa nút E khỏi Q[].

Q []

Kiểm tra nút E đã duyệt nên bỏ qua các phần sau, lặp lại **while**.

Kiểm tra, Q Rỗng nên kết thúc vòng **while**

Kết quả, dãy bfs có :

Thứ tự duyệt	1	2	3	4	5	6
Số hiệu nút	A	B	C	D	F	E

Ta có thể bắt đầu duyệt lại đồ thị theo bề rộng với một nút khác bất kỳ B/C/D/E/F.

Danh sách kết quả bfs[] chứa thứ tự duyệt đồ thị bề rộng từ nút bắt đầu sẽ cho một cây nhị phân đặc biệt là **Cây khung - Spanning Tree**, có tính chất : là một tập con của Graph G mà có tất cả các nút được bao bởi số cạnh tối thiểu, một **cây khung** sẽ không hình thành một vòng tuần hoàn và nó cũng không thể bị ngắt giữa chừng

Khi ta bỏ các cạnh có nét đứt, đồ thị kết quả là **cây khung** (spanning tree):

Đồ thị kết quả có tính chất :

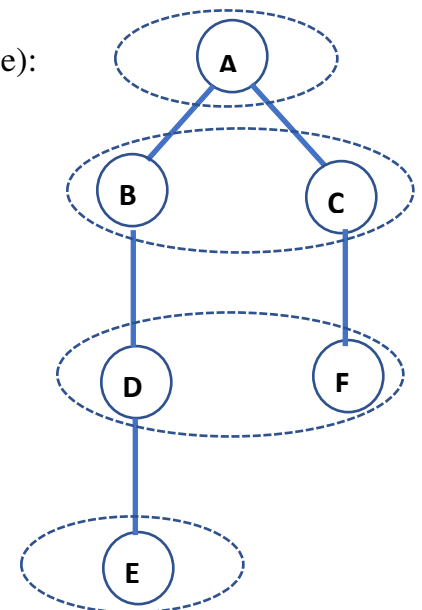
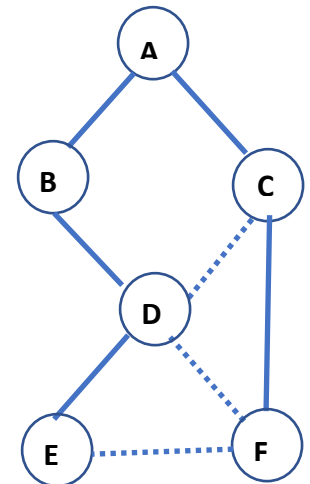
Mỗi nút của đồ thị kết quả

có thể đi đến từ nút khởi đầu khi

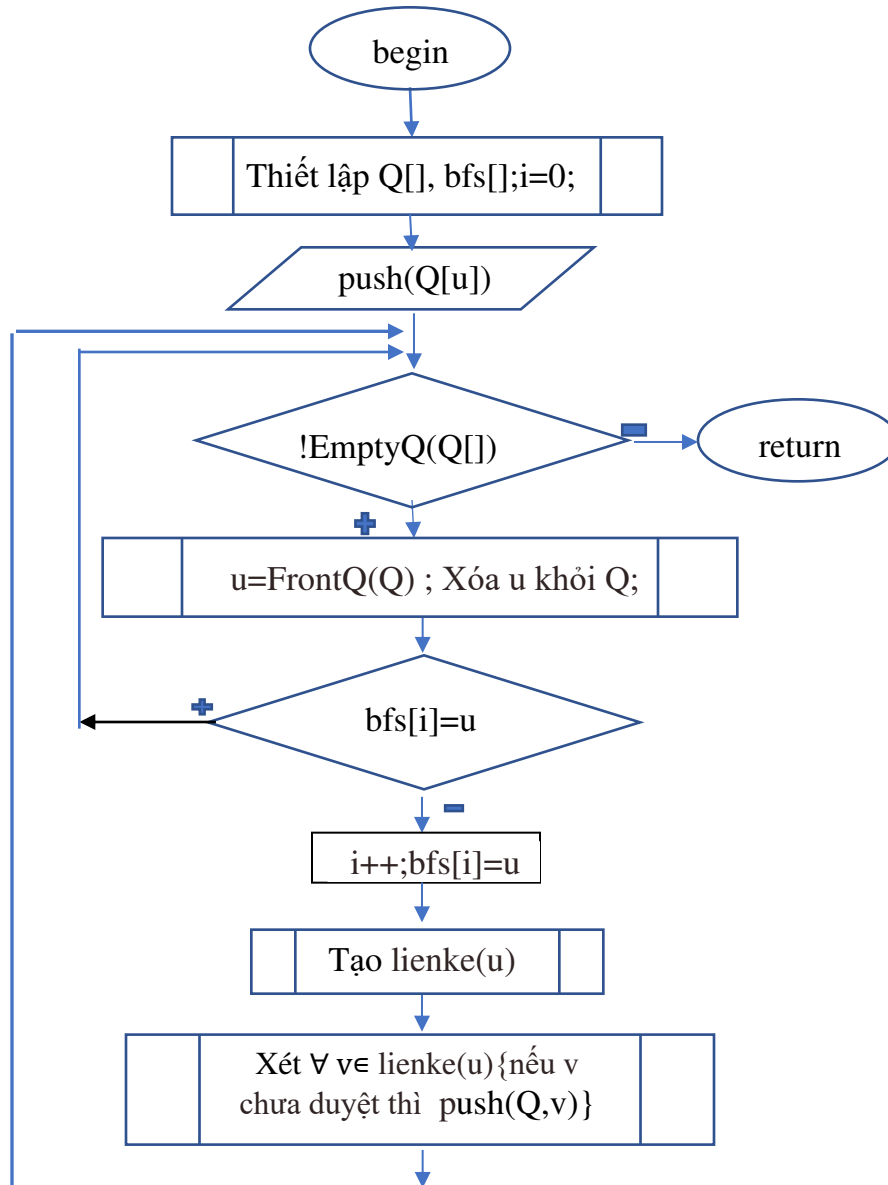
và chỉ khi nó được đánh dấu “đã duyệt”

Duyệt đồ thị theo BFS sẽ cho ta hình ảnh

duyet cây theo từng mức, từ mức 1 đến mức cuối cùng, mức sau cách mức trước 1 cạnh.



4.4.1.3. Sơ đồ thuật toán BFS :



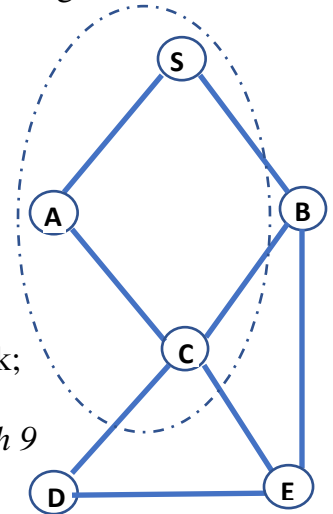
4.4.2. Thuật toán duyệt đồ thị theo chiều sâu – DFS

Duyệt đồ thị theo chiều sâu DFS gần giống với kiểu duyệt cây nhị phân DLR, luôn đi từ nút xuất phát tới nút xa nhất cho tới khi không đi tiếp được nữa thì quay lại nút trước đó.

1) Tổ chức dữ liệu : ở đây sử dụng một mảng dfs[] lưu các nút đã được duyệt và một ngăn xếp Stack phục vụ cho quá trình duyệt. Trên mỗi bước lặp chọn một nút bất kỳ trong các nút liền kề với nút đang xét, lặp lại việc này cho đến lúc không đi tiếp được nữa; sau đó trở lại nút trước đó và tìm đường sang nút khác nếu còn. Khi một nút mà các nút kề nó không đi tiếp được nữa và đã đánh dấu duyệt rồi thì sẽ xóa nó khỏi ngăn xếp Stack. Thuật toán kết thúc khi Stack đã rỗng.

2) Các bước tổng quát của thuật toán DFS:

- Đưa nút khởi đầu s vào ngăn xếp Stack và vào dfs[];
- while(!Empty(stack)){
 - w = lấy ra nút ở đỉnh ngăn xếp;
 - if(w đã duyệt) break; //thoát ra trở về while
 - Duyệt w; //thực hiện một thao tác nào đó với w;
 - Đánh dấu w đã duyệt : đưa w vào danh sách dfs[];
 - Tạo tập các nút liền kề của w : V(w);
 - for (v ∈ V(w); v chưa được duyệt-chưa có trong dfs[]) đưa v vào Stack;



Hình 9

3) Vận dụng vào bài toán cụ thể

Cho đồ thị như hình 9.

Ta tiến hành các bước với đồ thị như sau :

Khởi tạo mảng dfs :

i			
dfs[i].Tên nút			
dfs[i].Thứ tự duyệt			

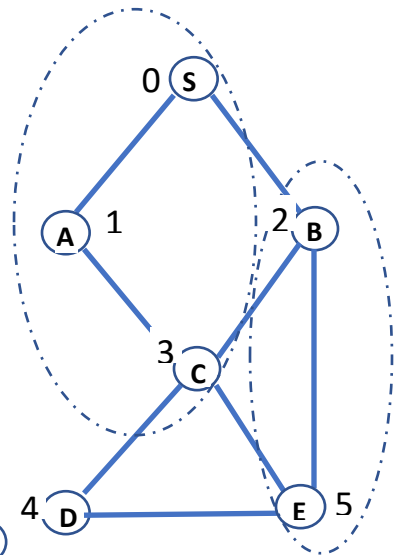
Và ngăn xếp stack

Có thể bắt đầu duyệt từ một nút bất kỳ, ở đây ta chọn nút S(start).

Đặt i là chỉ số của mảng dfs[], mỗi nút dfs[i] có 2 thành phần là Tên_nút và Thứ tự duyệt; đặt k là đếm số lần lặp DFS.

Bắt đầu từ i= 0 ; gán dfs[0].Tên_nút= S ; dfs[0].Thứ_tự_duyệt= 1.

k= 0; Đưa nút S vào stack



Lần lặp while thứ 1(k++=1):

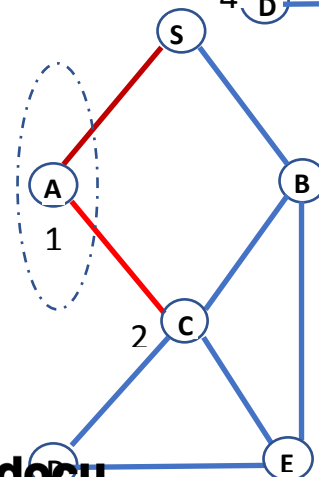
Lấy đỉnh ngăn xếp là S, xét tập nút liền kề của S = {A,B}, chọn nút liền kề đầu tiên mà chưa duyệt đưa vào stack : ở đây lấy A, dfs[1].Tên_nút= A, dfs[1].Thứ_tự_duyệt= 2, đưa nút A vào stack. Kiểm tra, ngăn xếp chưa rỗng, lặp tiếp

A
S

Lần lặp while thứ 2 (k++=2)

Lấy A- đỉnh Stack, lập tập nút liền kề của A={S,C}, đưa nút liền kề đầu tiên chưa duyệt là C vào Stack, i++, dfs[2].Tên_nút= C, dfs[2].Thứ_tự_duyệt= 3, Kiểm tra, ngăn xếp chưa rỗng, lặp tiếp

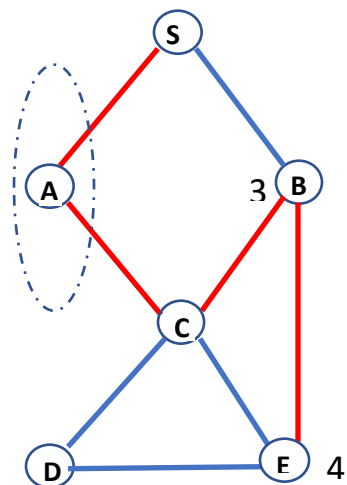
C
A
S



Lần lặp while thứ 3 ($k++=3$):

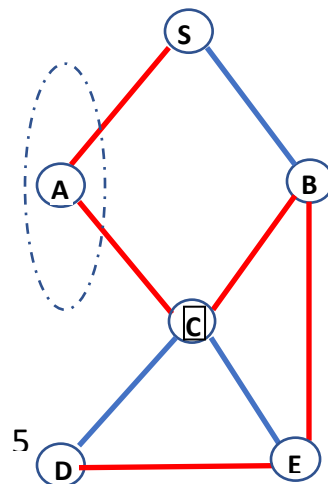
Lấy ra đỉnh ngăn xếp là C, xét tập nút liền kề của C = {A, B, E, D}. Chọn nút liền kề đầu tiên chưa duyệt là B đưa vào Stack. $i++$, $dfs[3].Tên_nút = B$, $dfs[3].Thứ_tự_duyet = 4$. Kiểm tra, ngăn xếp chưa rỗng, lặp tiếp

B
C
A
S

**Lần lặp while thứ 4 ($k++=4$)**

Lấy B- đỉnh ngăn xếp. Xét tập nút liền kề của B = {S, C, E}, Chọn nút liền kề đầu tiên chưa duyệt là E đưa vào Stack. $i++$, $dfs[4].Tên_nút = E$, $dfs[4].Thứ_tự_duyet = 5$. Kiểm tra, ngăn xếp chưa rỗng, lặp tiếp

E
B
C
A
S

**Lần lặp while thứ 5 ($k++=5$)**

Lấy E- đỉnh ngăn xếp. Xét tập nút liền kề của E = {B, C, D}, Chọn nút liền kề đầu tiên chưa duyệt là D đưa vào Stack. $i++$, $dfs[5].Tên_nút = D$, $dfs[5].Thứ_tự_duyet = 6$. Kiểm tra, ngăn xếp chưa rỗng, lặp tiếp

D
E
B
C
A
S

Lần lặp while thứ 6 ($k++=6$)

Lấy ra đỉnh ngăn xếp là D. Xét tập nút liền kề của D = {C, E}, Tất cả các nút liền kề với D đều đã duyệt nên xóa D ra khỏi stack. Kiểm tra, ngăn xếp chưa rỗng, lặp tiếp

E
B
C
A
S

Lần lặp while thứ 7 ($k++=7$):

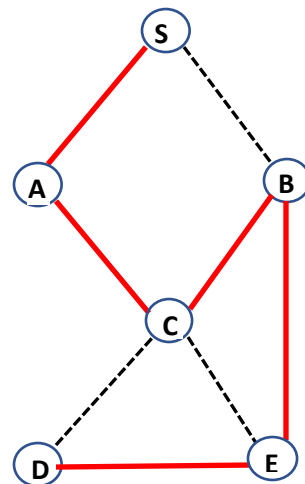
Lấy ra đỉnh ngăn xếp là E. Tất cả các nút liền kề với E đều đã duyệt nên xóa E ra khỏi Stack. Kiểm tra, ngăn xếp chưa rỗng, lặp tiếp

B
C
A
S

Lần lặp while thứ 8 ($k++=8$):

Lấy ra đỉnh ngăn xếp là B. Tất cả các nút liền kề với B đều đã duyệt nên xóa B ra khỏi Stack. Kiểm tra, ngăn xếp chưa rỗng, lặp tiếp

C
A
S

**Lần lặp while thứ 9 ($k++=9$)**

Lấy ra đỉnh ngăn xếp là C. Tất cả các nút liền kề với C đều đã duyệt nên xóa C ra khỏi Stack. Kiểm tra, ngăn xếp chưa rỗng, lặp tiếp

A
S

Lần lặp while thứ 10 ($k++=10$):

Lấy ra đỉnh ngăn xếp là A. Tất cả các nút liên kề với A đều đã duyệt nên xóa A ra khỏi Stack
Kiểm tra, ngăn xếp chưa rỗng, lặp tiếp

S

Lần lặp while thứ 11 ($k++=11$):

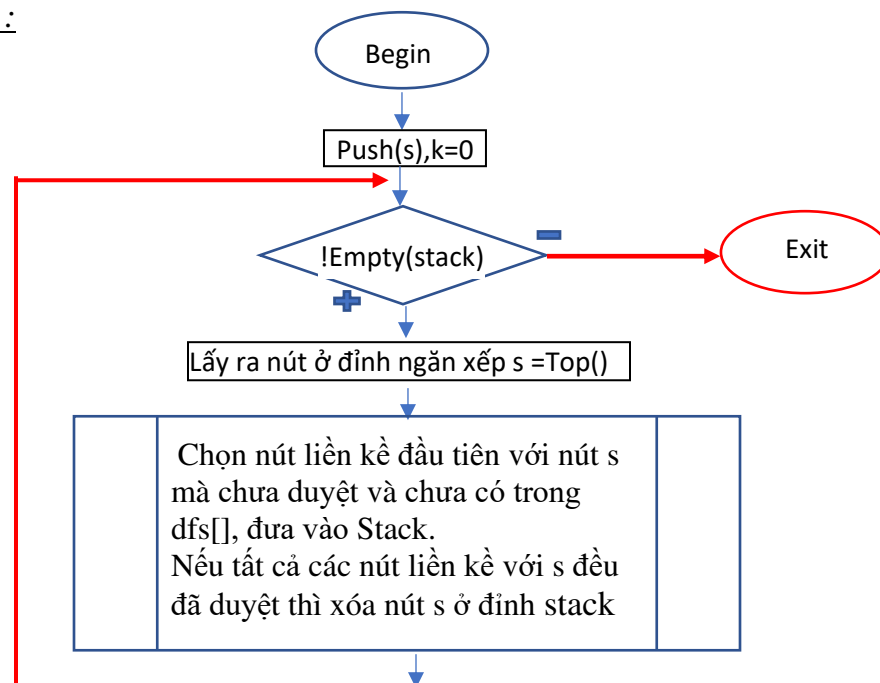
Lấy ra đỉnh ngăn xếp là S. Tất cả các nút liên kề với S đều đã duyệt nên xóa S ra khỏi stack
Kiểm tra, ngăn xếp rỗng, kết thúc lặp

Mảng dfs[] có kết quả là :

i =	0	1	2	3	4	5
dfs[i].dulieu =	S	A	C	B	E	D
dfs[i].ThuTuDuyet =	1	2	3	4	5	6

Cũng giống như duyệt theo bề ngang, khi duyệt theo chiều sâu ta có thể bắt đầu từ một nút bất kỳ.
Kết quả duyệt sẽ cho một **cây khung** (spanning tree) S-A-C-B-E-D :

Sơ đồ thuật toán DFS :



Thuật toán có thể bắt đầu bằng một nút bất kỳ.

Chú ý : trong các sơ đồ khối thuật toán , khối ký hiệu một quá trình gồm nhiều bước hay cả một khối các phép toán mà khi lập trình tạo thành một hàm.

Bài tập. 1) Đọc, tìm hiểu thuật toán duyệt đồ thị theo bề ngang và theo chiều sâu.

2) Cho đồ thị :

Hãy trình bày từng bước duyệt đồ thị theo bề ngang và theo chiều sâu, có thể bắt đầu từ một nút bất kỳ.

